

To compile and run all tests type “mvn package”

In order to run my code, use the following syntax:

1. First create a MyGraph object by writing
 - a. `MyGraph myg = new MyGraph();`
 - i. Note that this creates a MutableGraph object from the graphviz library and stores it in the MyGraph's mutGraph attribute.
2. Now parse a .dot file named “ex.dot” using the parseGraph function like this
 - a. `myg.parseGraph(“ex.dot”);`
 - i. Note that this will throw a FileNotFoundException if the file does not exist.
3. You can convert a MyGraph to a string like this
 - a. `myg.toString();`
4. You can output the same string to a file named “ex.dot” like this
 - a. `myg.outputGraph(“ex.dot”);`
5. You can add a node named “a” to a MyGraph like this
 - a. `myg.addNode(“a”);`
 - i. Note that this will not add the node if the label already exists in the graph.
6. You can add an array of nodes named “a”, “b”, and “c” to a MyGraph like this
 - a. `nodes = {“a”, “b”, “c”};`
 - b. `myg.addNodes(nodes);`
 - i. Note that this will only add the nodes whose labels don't already exist in the graph.
7. You can add an edge to a MyGraph from node “a” to node “b” like this
 - a. `myg.addEdge(“a”, “b”);`
 - i. Note that this will only add the edge if both nodes already exist in the graph.
8. You can send your MyGraph to a .dot file named “ex.dot” like this
 - a. `myg.outputDOTGraph(“ex.dot”);`
9. You can send your MyGraph to a png file named “ex.png” like this
 - a. `myg.outputGraphics(“ex”, “png”);`
 - i. Note: this only works for png files.

The following syntax is for part 2:

10. You can remove a node from your graph like this
 - a. `myg.removeNode(“ex”);`

- i. Note: If the node does not exist, it will throw an exception.
 - ii. Note: removing a node will automatically remove all edges linked to that node.
- 11. You can remove all nodes in an array of strings "list" like this:
 - a. `myg.removeNodes(list);`
 - i. Note: For each node that does not exist, it will throw an exception.
 - ii. Note: removing a node will automatically remove all edges linked to that node.
- 12. You can remove an edge from "a" to "b" like this:
 - a. `myg.removeEdge("a", "b");`
 - i. Note: if the edge doesn't exist, it will throw an exception.
- 13. You can search a graph from source node "a" to destination node "b" like this:
 - a. `myg.graphSearch("a", "b", Algorithm.BFS);`
 - i. Note: the function returns a Path object that has a String array attribute which represents the path from "a" to "b".
 - ii. Note: the third parameter is an enum that tells the function which algorithm to use. The options are `Algorithm.BFS` and `Algorithm.DFS`. these run breadth first search and depth first search respectively.
 - iii. Note: if no path from node "a" to "b" exists, the function will return null.

In order to test my code, type "mvn test"

Here is a breakdown of all the test cases in `MyGraphTest.java`. All testing .dot files can be found in the github repository. Each test will have a description, screenshot of code, and screenshot of output.

1. The `setup()` function creates a `MyGraph` object that all test cases can use. It has the `@BeforeEach` tag meaning that a new `MyGraph` is created before each test is run.

```
MyGraph myg;  
  
@BeforeEach  
public void setup(){  
    myg = new MyGraph();  
}
```

No output created

2. The `testParseGraph()` function tests the `parseGraph()` function by parsing the file "ParseInput.dot". The function also creates a `MutableGraph` object from the `graphviz` library. It then adds nodes and edges to this `MutableGraph` such that it is equivalent to

the graph represented in "ParseInput.dot". It then asserts that both graphs are equivalent by using the graphviz toString() function.

```
@Test
public void testParseGraph() throws IOException {
    //arrange
    MutableGraph mutg = mutGraph().setDirected(true);
    MutableNode a = mutNode("a");
    MutableNode b = mutNode("b");
    MutableNode c = mutNode("c");
    MutableNode d = mutNode("d");

    a.addLink(b);
    b.addLink(c);
    c.addLink(d);

    mutg.add(a);
    mutg.add(b);
    mutg.add(c);
    mutg.add(d);

    //act
    myg.parseGraph("ParseInput.dot");

    //assert
    Assertions.assertEquals(mutg.toString(), myg.mutGraph.toString());
}
```

```
digraph {
  "a" -> "b"
  "b" -> "c"
  "c" -> "d"
}
```

3. The testParseGraphNoFile() function tests that the parseGraph() function throws a FileNotFoundException when the input file that it is given does not exist.

```
@Test
public void testParseGraphNoFile(){
    Assertions.assertThrows(FileNotFoundException.class, () -> {
        myg.parseGraph("NotAFile.txt");
    });
}
```

```

at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
at org.junit.jupiter.api.AssertThrows.assertThrows(AssertThrows.java:67)
at org.junit.jupiter.api.AssertThrows.assertThrows(AssertThrows.java:35)
at org.junit.jupiter.api.Assertions.assertThrows(Assertions.java:3115)
at MyGraphTest.testParseGraphNoFile(MyGraphTest.java:49)
at java.base/java.lang.reflect.Method.invoke(Method.java:580)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
Caused by: java.io.FileNotFoundException: Create breakpoint : NotAFile.txt (The system cannot find the file specified)
at java.base/java.io.FileInputStream.open0(Native Method)
at java.base/java.io.FileInputStream.open(FileInputStream.java:219)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:159)
at guru.nidi.graphviz.parse.Parser.read(Parser.java:68)
at MyGraph.parseGraph(MyGraph.java:24)
at MyGraphTest.lambda$testParseGraphNoFile$0(MyGraphTest.java:50)
at org.junit.jupiter.api.AssertThrows.assertThrows(AssertThrows.java:53)
... 6 more

```

4. The testParseGraphSyntaxError() function tests that the parseGraph() function throws a ParserException when the input file that it is given does not fit the dot syntax.

```

@Test
public void testParseGraphSyntaxError() {
    Assertions.assertThrows(ParserException.class, () -> {
        myg.parseGraph("ParseSyntaxErrorInput.dot");
    });
}

```

```

at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
at org.junit.jupiter.api.AssertThrows.assertThrows(AssertThrows.java:67)
at org.junit.jupiter.api.AssertThrows.assertThrows(AssertThrows.java:35)
at org.junit.jupiter.api.Assertions.assertThrows(Assertions.java:3115)
at MyGraphTest.testParseGraphSyntaxError(MyGraphTest.java:56)
at java.base/java.lang.reflect.Method.invoke(Method.java:580)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
Caused by: ParseSyntaxErrorInput.dot:1:8 'graph' or 'digraph' expected.
at guru.nidi.graphviz.parse.ParserImpl.fail(ParserImpl.java:331)
at guru.nidi.graphviz.parse.ParserImpl.lambda$parse$0(ParserImpl.java:59)
at guru.nidi.graphviz.model.ThrowingFunction.applyNotThrowing(ThrowingFunction.java:24)
at guru.nidi.graphviz.model.CreationContext.use(CreationContext.java:44)
at guru.nidi.graphviz.model.CreationContext.use(CreationContext.java:38)
at guru.nidi.graphviz.parse.ParserImpl.parse(ParserImpl.java:50)
at guru.nidi.graphviz.parse.Parser.read(Parser.java:81)
at guru.nidi.graphviz.parse.Parser.read(Parser.java:68)
at MyGraph.parseGraph(MyGraph.java:24)
> at MyGraphTest.lambda$testParseGraphSyntaxError$1(MyGraphTest.java:57) <1 internal line>
... 6 more

```

5. The testToString() function tests that the toString() function works by creating a string that is the expected output of the string format of the dot graph in the "GeneralInput.dot" file. It then asserts that these are equal.

```

@Test
public void testToString() throws IOException {
    myg.parseGraph("GeneralInput.dot");

    String str = "digraph {\n" +
        "\"a\" -> \"b\"\n" +
        "\"b\" -> \"c\"\n" +
        "\"c\" -> \"d\"\n" +
        "}";

    Assertions.assertEquals(str, myg.toString());
}

```

```

digraph {
"a" -> "b"
"b" -> "c"
"c" -> "d"
}

```

6. The testOutputGraph() function tests that the outputGraph() function works by parsing the "generalInput.dot" file and then outputting the graph to "Output.dot". The function then asserts that the lines in the output file are equal to the lines in the "GeneralExpected.dot" file.

```

@Test
public void testOutputGraph() throws IOException {
    myg.parseGraph("GeneralInput.dot");

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("GeneralExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

```

```

digraph {
"a" -> "b"
"b" -> "c"
"c" -> "d"
}

```

7. The testAddNode() function tests that the addNode() function works by adding a node "a" to the graph and then outputting the graph to "Output.dot". The function then asserts that the lines in the output file are equal to the lines in the "addNodeExpected.dot" file.

```

@Test
public void testAddNode() throws IOException {
    myg.addNode("a");

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("AddNodeExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

```

```

digraph {
    "a"
}

```

8. The testAddDuplicateNode() function tests that the addNode() function works by adding a node “a” to the graph twice and then outputting the graph to “Output.dot”. The function then asserts that the lines in the output file are equal to the lines in the “addNodeExpected.dot” file.

```

@Test
public void testAddDuplicateNode() throws IOException {
    myg.addNode("a");
    myg.addNode("a");

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("AddNodeExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

```

```

digraph {
    "a"
}

```

9. The testAddNodes() function tests that the addNodes() function works by adding a list of nodes “a”, “b”, “c” to the graph and then outputting the graph to “Output.dot”. The function then asserts that the lines in the output file are equal to the lines in the “addNodesExpected.dot” file.

```

@Test
public void testAddNodes() throws IOException {
    String[] labels = {"a", "b", "c"};
    myg.addNodes(labels);

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("AddNodesExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

```

```

digraph {
    "a"
    "b"
    "c"
}

```

10. The testAddEdge() function tests that the addEdge() function works by adding nodes “a” and “b” and then an edge from “a” to “b” to the graph and then outputting the graph to “Output.dot”. The function then asserts that the lines in the output file are equal to the lines in the “addEdgeExpected.dot” file.

```

@Test
public void testAddEdge() throws IOException {
    myg.addNode("a");
    myg.addNode("b");

    myg.addEdge("a", "b");

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("AddEdgeExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

```

```

digraph {
    "a" -> "b"
}

```

11. The testAddEdgeNoNodes() function tests that the addEdge() function works by adding an edge from “a” to “b” to the graph while there is no nodes. Then it outputs the graph to “Output.dot”. The function then asserts that the lines in the output file are equal to the lines in the “addEdgeNoNodesExpected.dot” file.

```

@Test
public void testAddEdgeNoNodes() throws IOException {
    myg.addEdge("a", "b");

    myg.outputGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("AddEdgeNoNodesExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

digraph {
}

```

12. The testOutputDOTGraph() function tests that the outputDOTGraph() function works by parsing the "GeneralInput.dot" file and then outputting the graph to "Output.dot". The function then asserts that the lines in the output file are equal to the lines in the "GeneralExpected.dot" file.

```

@Test
public void testOutputDOTGraph() throws IOException {
    myg.parseGraph("GeneralInput.dot");

    myg.outputDOTGraph("Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("GeneralExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}

digraph {
    "a" -> "b"
    "b" -> "c"
    "c" -> "d"
}

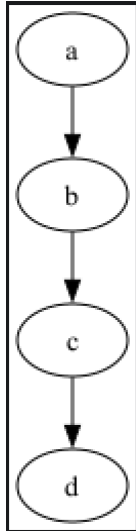
```

13. The testOutputGraphics() function tests that the outputGraphics() function works by parsing the "GeneralInput.dot" file and then outputting the graph to "Output.png". The function then asserts that the output file exists.


```
@Test
public void testOutputGraphics() throws IOException {
    myg.parseGraph("GeneralInput.dot");

    myg.outputGraphics("Output", "png");

    Assertions.assertTrue(Files.exists(Path.of("Output.png")));
}
```



14. The `testOutputGraphicsExtension()` function tests that the `outputGraphics()` function works by parsing the “GeneralInput.dot” file and then outputting the graph to “Output.png”. The function then asserts that the output file has the correct file type.

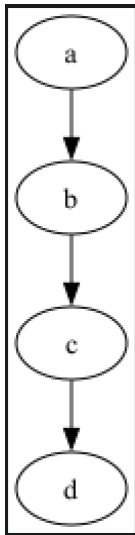
```

@Test
public void testOutputGraphicsExtension() throws IOException {
    myg.parseGraph("GeneralInput.dot");

    myg.outputGraphics("Output", "png");
    Path p = Path.of("Output.png");
    String type = Files.probeContentType(p);

    Assertions.assertEquals("image/png", type);
}

```



The following test functions are for part 2:

15. The testRemoveNode() function tests that the removeNode() function works by parsing the "GeneralInput.dot" file then removing node "a" and outputting the graph to "Output.dot". The function then asserts that the lines in the output file are equal to the lines in the "RemoveNodeExpected.dot" file.

```

@Test
public void testRemoveNode() throws Exception {
    myg.parseGraph(filepath: "GeneralInput.dot");

    myg.removeNode(label: "a");

    myg.outputGraph(filepath: "Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of(first: "RemoveNodeExpected.dot")), Files.readAllLines(Path.of(first: "Output.dot")));
}

```

```
strict digraph G {
    b;
    c;
    d;
    b -> c;
    c -> d;
}
```

16. The testRemoveNodeEmpty() function tests that the removeNode() function works by attempting to remove node “a” from an empty graph. The function then asserts that there was an exception thrown and that the exception message is expected.

```
@Test hgkko
public void testRemoveNodeEmpty() throws Exception {
    Exception e = Assertions.assertThrows(Exception.class, () -> {
        myg.removeNode(label: "a");
    });

    Assertions.assertEquals(expected: "Node a does not exist", e.getMessage());
}
```

Node a does not exist

17. The testRemoveNodes() function tests that the removeNodes() function works by parsing the “GeneralInput.dot” file then removing nodes from a list {“a”, “b”} and outputting the graph to “Output.dot”. The function then asserts that the lines in the output file are equal to the lines in the “RemoveNodesExpected.dot” file.

```
@Test hgkko
public void testRemoveNodes() throws Exception {
    myg.parseGraph(filepath: "GeneralInput.dot");

    String[] label = {"a", "b"};
    myg.removeNodes(label);

    myg.outputGraph(filepath: "Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of("RemoveNodesExpected.dot")), Files.readAllLines(Path.of("Output.dot")));
}
```

```
strict digraph G {
    c;
    d;
    c -> d;
}
```

18. The testRemoveEdge() function tests that the removeEdge() function works by parsing the “GeneralInput.dot” file then removing the Edge from a “a” -> “b” and outputting the

graph to "Output.dot." The function then asserts that the lines in the output file are equal to the lines in the "RemoveEdgeExpected.dot" file.

```
@Test  hgkko
public void testRemoveEdge() throws Exception {
    myg.parseGraph( filepath: "GeneralInput.dot");

    myg.removeEdge( srcLabel: "a", dstLabel: "b");

    myg.outputGraph( filepath: "Output.dot");

    Assertions.assertEquals(Files.readAllLines(Path.of( first: "RemoveEdgeExpected.dot")), Files.readAllLines(Path.of( first: "Output.dot")));
}
```

```
strict digraph G {
    a;
    b;
    c;
    d;
    b -> c;
    c -> d;
}
```

19. The testRemoveEdgeEmpty() function tests that the removeEdge() function works by attempting to remove edge "a" -> "b" from an empty graph. The function then asserts that there was an exception thrown and that the exception message is expected.

```
@Test  hgkko
public void testRemoveEdgeEmpty(){
    Exception e = Assertions.assertThrows(Exception.class, () -> {
        myg.removeEdge( srcLabel: "a", dstLabel: "b");
    });

    Assertions.assertEquals( expected: "Edge a -> b does not exist", e.getMessage());
}
```

```
Edge a -> b does not exist
```

20. The testGraphSearchBFS() function tests that the graphSearch() function works by parsing the "GeneralInput.dot" file then searching for a path from node "a" to node "c" using breadth first search. The function then asserts that the returned Path objects array attribute is equal to the expected.

```

@Test hgkko
public void testGraphSearchBFS() throws IOException {
    myg.parseGraph( filepath: "GeneralInput.dot");

    String[] p = myg.graphSearch( src: "a", dst: "c", Algorithm.BFS).p;

    String[] a = new String[16];
    a[0] = "a";
    a[1] = "b";
    a[2] = "c";

    Assertions.assertArrayEquals(a, p);
}

```

```

{"a", "b", "c"}

```

21. The testGraphSearchDFS() function tests that the graphSearch() function works by parsing the "GeneralInput.dot" file then searching for a path from node "a" to node "c" using depth first search. The function then asserts that the returned Path objects array attribute is equal to the expected.

```

@Test hgkko
public void testGraphSearchBFS() throws IOException {
    myg.parseGraph( filepath: "GeneralInput.dot");

    String[] p = myg.graphSearch( src: "a", dst: "c", Algorithm.BFS).p;

    String[] a = new String[16];
    a[0] = "a";
    a[1] = "b";
    a[2] = "c";

    Assertions.assertArrayEquals(a, p);
}

```

```

{"a", "b", "c"}

```

Github commit links:

Commit 1:

<https://github.com/HKosten/CSE464CourseProject1/commit/422721e20e893a215c94db9d92024e88b821d61e>

Commit 2:

<https://github.com/HKosten/CSE464CourseProject1/commit/03fea3671092ddeb5634412acb780a8e849ff642>

Commit 3:

<https://github.com/HKosten/CSE464CourseProject1/commit/1327a7e770de7d45b33ed4c94c3516cd93150b12>

Commit 4:

<https://github.com/HKosten/CSE464CourseProject1/commit/6f285c93005f6bd61aa45ab53e25a9cd07a91a3c>

The following commits are for part 2:

Commit 5: Removing nodes and edges

<https://github.com/HKosten/CSE464CourseProject1/commit/35d5162865a6efe087a339e7c41bee35000a0ded>

Commit 6: Continuous integration file

<https://github.com/HKosten/CSE464CourseProject1/commit/a98e25543dbeb758da8afdd3d6a73c134ab80c59>

Commit 7: Missing test file to enable CI tests.

<https://github.com/HKosten/CSE464CourseProject1/commit/1a7f9192be80b516fd273503dce9fa15de50c947>

Commit 8: Breadth-first search branch

<https://github.com/HKosten/CSE464CourseProject1/commit/eb3d4b6e61182e2283991a8480ddccfc86d25a9f>

Commit 9: Depth-first search branch

<https://github.com/HKosten/CSE464CourseProject1/commit/d5b76bfa1bd4bee044d8869c6fc352e079f3a231>

Commit 10: Merge DFS

<https://github.com/HKosten/CSE464CourseProject1/commit/cb8f3c5d7da1d59f517f78b2900fb5fe5dece4cc>

Commit 11: Merge BFS and create algo parameter

<https://github.com/HKosten/CSE464CourseProject1/commit/cf0777db393de9c1d209059cb77c7536b2a7c780>